

Relatório Técnico de Segurança

Esteira DevSecOps com GitHub Actions

Projeto *Health In Sight*

Repositório	Stack	Entrega
github.com/Guilheslherme/ projeto_politicas_seguranca	Python 3 / Django 5.2 MySQL · Unicorn	Outubro / 2026 Repositório Público

1 · Introdução e Contextualização

O modelo tradicional de desenvolvimento de software trata a segurança como uma etapa final, executada após a conclusão do produto. Essa abordagem gera ciclos longos de correção, custos elevados e exposição contínua a vulnerabilidades. O paradigma **DevSecOps** (Development + Security + Operations) inverte essa lógica ao integrar controles de segurança diretamente no pipeline de integração contínua — princípio conhecido como *Shift Left Security*.

Shift Left: mover as verificações de segurança para o início do ciclo de desenvolvimento, interceptando falhas antes que cheguem à produção — reduzindo custo e risco de forma significativa.

Neste projeto, implementamos uma esteira automatizada no **GitHub Actions** composta por três vertentes complementares de segurança: varredura de segredos expostos, análise de vulnerabilidades em dependências e análise estática do código-fonte. A cada *push* ou *pull request* na branch `main`, o pipeline é acionado automaticamente e bloqueia o merge caso encontre falhas de severidade **HIGH** ou **CRITICAL**.

2 · Descrição das Ferramentas Adotadas

2.1 · TruffleHog — Secrets Scanning

O **TruffleHog** é uma ferramenta open-source especializada em detectar segredos (chaves de API, senhas, tokens, chaves criptográficas) expostos acidentalmente no histórico de commits do repositório Git.

Aspecto	Detalhe
Desenvolvedor	Truffle Security Co.
Método de detecção	Análise de entropia + regex de padrões conhecidos + verificação ativa
Flag utilizada	<code>--only-verified</code> : reporta apenas segredos confirmados como válidos
Integração	GitHub Action oficial: <code>trufflesecurity/trufflehog@main</code>
Disparador	Automático no contexto do push (<code>github.event.before</code> → <code>github.sha</code>)

Justificativa: O projeto armazena variáveis sensíveis como `DJANGO_SECRET_KEY` , `FIELD_ENCRYPTION_KEY` (AES-256) e credenciais de banco de dados no arquivo `.env` . Um commit acidental desse arquivo poderia expor dados críticos. O TruffleHog garante que isso seja detectado imediatamente, antes de qualquer merge na branch principal.

2.2 · Trivy — SCA (Software Composition Analysis)

O **Trivy**, desenvolvido pela Aqua Security, é um scanner de vulnerabilidades abrangente que analisa dependências de projetos (entre outras superfícies) em busca de CVEs catalogados na NVD (National Vulnerability Database) e OSV.

Aspecto	Detalhe
Desenvolvedor	Aqua Security
Modo de varredura	<code>fs</code> — detecta <code>requirements.txt</code> automaticamente
Severidades monitoradas	HIGH e CRITICAL

Aspecto	Detalhe
Política de bloqueio	<code>exit-code: "1"</code> — interrompe o pipeline em falhas críticas
Integração	GitHub Action: <code>aquasecurity/trivy-action@master</code>

Dependências do projeto analisadas:

Pacote	Versão	Relevância de segurança
Django	5.2.17	Framework principal — CVEs de XSS, CSRF e bypass de autenticação são comuns
cryptography	50.0.1	Usado para cifrar o segredo 2FA (AES-256) — vulnerabilidades podem comprometer dados em repouso
Pillow	12.3.0	Processamento de imagens — histórico de CVEs de heap overflow
gunicorn	26.2.0	Servidor WSGI em produção — HTTP Request Smuggling é um vetor crítico
mysqlclient	2.2.8	Driver de banco de dados — falhas podem levar a SQL Injection

2.3 · GitHub CodeQL — SAST (Static Application Security Testing)

O **CodeQL** é a ferramenta nativa de análise semântica do GitHub. Diferente de ferramentas baseadas em padrões de texto, o CodeQL constrói um modelo do código-fonte (grafo de fluxo de dados) e detecta vulnerabilidades complexas que só se manifestam ao longo de múltiplos arquivos e chamadas de função.

Aspecto	Detalhe
Desenvolvedor	GitHub (Microsoft)
Linguagem analisada	Python
Query suite	<code>security-and-quality</code>
Resultado	Alertas publicados na aba <i>Security</i> → <i>Code scanning</i> do repositório

Aspecto	Detalhe
Permissão necessária	<code>security-events: write</code>
Disponibilidade	Gratuito para repositórios públicos — opção escolhida por ser repositório público

Vulnerabilidades detectáveis no contexto Django:

- SQL Injection via queries brutas (`raw()` , `extra()`)
- Path Traversal em upload de arquivos
- Server-Side Template Injection (SSTI)
- Uso inseguro de funções criptográficas (ex.: MD5 para dados sensíveis)
- Desserialização insegura

Por que CodeQL e não Semgrep? A instrução da atividade permite explicitamente o uso do CodeQL como opção bônus para repositórios públicos, aproveitando a análise semântica nativa do GitHub. O CodeQL oferece detecção mais profunda de fluxos de dados maliciosos do que o Semgrep open-source.

3 · Integração com o Documento de Requisitos

O documento *Health-In-Sight-Documentacao-Tecnico-Cientifica.pdf* estabelece requisitos de segurança que a esteira DevSecOps auxilia a monitorar e garantir continuamente:

Requisito do Projeto	Ferramenta na Esteira	Como protege
Chave AES-256 para cifrar segredos do 2FA em repouso (<code>FIELD_ENCRYPTION_KEY</code>)	TruffleHog	Detecta se a chave for commitada acidentalmente no histórico Git
Django Secret Key nunca exposta (<code>DJANGO_SECRET_KEY</code>)	TruffleHog	Varre todos os commits em busca de padrões de chave Django
Credenciais do banco MySQL protegidas (<code>DB_PASSWORD</code>)	TruffleHog	Identifica strings de conexão e senhas expostas

Requisito do Projeto	Ferramenta na Esteira	Como protege
Dependências atualizadas e livres de CVEs conhecidos	Trivy SCA	Bloqueia merge se pacotes como Django, Pillow ou cryptography tiverem CVE HIGH/CRITICAL
Código livre de vulnerabilidades de injeção e manipulação de dados	CodeQL SAST	Analisa o fluxo de dados no código Python e detecta SQL Injection, Path Traversal, etc.
Autenticação de dois fatores (2FA) com <code>django-otp</code>	CodeQL SAST	Detecta bypasses lógicos na implementação do fluxo de autenticação
Bloqueio após tentativas falhas (<code>django-axes</code>)	CodeQL SAST	Verifica implementação correta de controles de acesso e lógica de bloqueio

Proteção contínua: A cada novo commit, a esteira revalida todos os requisitos de segurança acima de forma automática. Qualquer regressão (ex.: remoção de um controle, adição de uma dependência vulnerável) é detectada antes de chegar à produção.

4 · Evidências de Funcionamento

4.1 · Arquivo de Pipeline (devsecops-pipeline.yml)

```
name: DevSecOps Pipeline

on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]

jobs:
  secrets-scanning:
    name: "🔑 Secrets Scanning (TruffleHog)"
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: trufflesecurity/trufflehog@main
        with:
          path: ./
          extra_args: --only-verified

  dependency-scan:
    name: "📦 Dependency Scan (Trivy SCA)"
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aquasecurity/trivy-action@master
        with:
          scan-type: fs
          scan-ref: .
          scanners: vuln
          severity: HIGH,CRITICAL
          exit-code: "1"
          ignore-unfixed: true
          format: table

  sast-codeql:
    name: "🔴 SAST (CodeQL)"
    runs-on: ubuntu-latest
    permissions:
      actions: read
      contents: read
      security-events: write
```

```
steps:
  - uses: actions/checkout@v4
  - uses: github/codeql-action/init@v3
    with:
      languages: python
      queries: security-and-quality
  - uses: github/codeql-action/autobuild@v3
  - uses: github/codeql-action/analyze@v3
    with:
      category: /language:python
```

4.2 · Execução da Esteira — GitHub Actions



Print da aba Actions — Pipeline rodando com os 3 jobs

Inserir captura de tela do GitHub Actions mostrando os jobs:

🔑 Secrets Scanning · 📦 Dependency Scan · 🧠 SAST (CodeQL) — todos em verde ✅

4.3 · Job: Secrets Scanning (TruffleHog)



Print do log do job TruffleHog

*Inserir captura mostrando o job concluído com sucesso (exit code 0)
e confirmação de que nenhum segredo verificado foi encontrado.*

4.4 · Job: Dependency Scan (Trivy SCA)



Print do log do job Trivy

*Inserir captura mostrando o relatório de vulnerabilidades do Trivy
e o status final do job (verde = sem CVEs HIGH/CRITICAL, ou vermelho = bloqueado).*

4.5 · Job: SAST — CodeQL



Print do log do job CodeQL + aba Security → Code scanning
Inserir captura mostrando o job concluído e, na aba Security do repositório, os alertas gerados (ou a ausência deles) pelo CodeQL.

4.6 · Política de Bloqueio — Comportamento Esperado

Cenário	Ferramenta	Comportamento do Pipeline
Segredo verificado no histórico	TruffleHog	✗ Job falha — exit code 1 — merge bloqueado
CVE HIGH ou CRITICAL em dependência	Trivy	✗ Job falha — exit code 1 — merge bloqueado
Vulnerabilidade no código Python	CodeQL	⚠ Alerta publicado na aba Security
Nenhuma falha encontrada	Todos	✓ Pipeline verde — merge liberado

5 · Aderência aos Critérios de Avaliação

Configuração do Pipeline	35%
Política de Bloqueio	25%
Qualidade do Relatório	40%

Critério	Requisito	Implementação
Configuração do Pipeline (35%)	YAML correto, sem erros, com 3 jobs	✓ 3 jobs implementados: TruffleHog + Trivy + CodeQL
Política de Bloqueio (25%)	Exit-code 1 em falhas HIGH/CRITICAL	✓ Trivy com <code>exit-code: "1"</code> · TruffleHog falha em segredos verificados
Qualidade do Relatório (40%)	Clareza técnica + prints + correlação com requisitos	✓ Seções 2, 3 e 4 atendem a este critério

Conclusão

A esteira DevSecOps implementada eleva o nível de maturidade de segurança do projeto *Health In Sight* ao automatizar três vertentes complementares: detecção de segredos expostos, análise de vulnerabilidades em dependências e inspeção estática do código-fonte.

O pipeline é ativado automaticamente a cada modificação na branch principal, garantindo que nenhuma alteração passe sem revisão de segurança. A política de bloqueio assegura que falhas de severidade alta ou crítica impeçam o avanço do código até a produção, reduzindo significativamente a janela de exposição a vulnerabilidades conhecidas.

Essa abordagem alinha-se diretamente aos requisitos de confidencialidade, integridade e disponibilidade descritos no documento de segurança do projeto, reforçando controles já existentes como autenticação 2FA, hashing Argon2 e bloqueio por tentativas falhas.